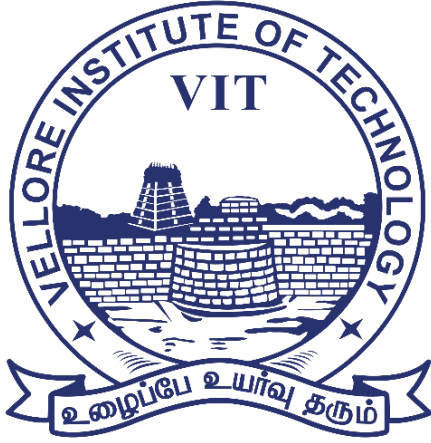




VIT[®]
B H O P A L
www.vitbhopal.ac.in

PROJECT:

Cab Dispatch & Surge Pricing Simulator

Subject: Programming in Java (CSE2006)

Name: Shubrat Jain

Registration No.: 25BAI10800

Faculty: Sanat Jain

PROJECT REPORT

Cab Dispatch & Surge Pricing Simulator

Report Contents

1. Introduction
2. Problem Statement
3. Objectives and Scope
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Decisions and Rationale
8. Implementation Details
9. Screenshots and Results
10. Testing Approach
11. Challenges Addressed
12. Future Enhancements
13. Conclusion
14. References

1. Introduction

On-demand cab platforms are software systems in which ride requests can arrive concurrently while the available driver pool is limited. Even in a small simulator, this produces a meaningful systems problem: the application has to maintain a consistent view of driver availability while several request handlers execute at the same time. The project models that problem in a compact Java application suitable for demonstrating core Programming in Java concepts.

The implementation separates domain entities from services, persistence, exceptions, worker threads, and tests. Main acts as the coordinator, RiderRequestThread models individual concurrent ride lifecycles, DriverPool manages the shared fleet, SurgePricingService calculates pricing, and RideDAO persists data to an embedded SQLite database. The repository also contains Maven configuration and platform-specific build scripts.

The project is deliberately a simulation rather than a full ride-hailing platform. It does not implement live maps, GPS routing, network services, authentication, or a graphical/web front end. Trip distance is provided by the rider model and the ride duration is simulated with Thread.sleep().

2. Problem Statement

The core problem is concurrent allocation of a constrained driver fleet. If multiple rider requests inspect the driver pool without synchronization, two threads can observe the same driver as AVAILABLE before either thread changes its status. This race condition can result in double-booking. The repository identifies this as the primary concurrency hazard the project is designed to address.

A second problem is pricing under demand spikes. With a static fare, the simulator would not represent changing demand pressure. The project therefore introduces a supply-and-demand multiplier so that prices increase when active requests exceed the currently available drivers, subject to a maximum multiplier of 3.00x.

The solution must therefore provide four core properties: (1) thread-safe driver assignment, (2) dynamic and bounded pricing, (3) graceful failure when no driver is available, and (4) persistent ride/audit records with a generated report.

3. Objectives and Scope

3.1 Objectives

- Apply Java multithreading to model simultaneous ride requests.
- Use synchronization to make driver allocation a mutually exclusive critical section.
- Implement a bounded surge-pricing algorithm based on current demand and supply.
- Demonstrate custom exception handling for invalid requests and fleet exhaustion.
- Persist driver and ride information using SQLite through JDBC.
- Generate a readable post-simulation audit report using Java I/O streams.
- Provide automated verification for key correctness properties.

3.2 Scope

In scope are rider-request threads, shared driver allocation, driver state transitions, surge calculation, ride lifecycle states, SQLite persistence, report generation, and automated tests. The repository deliberately avoids heavy frameworks such as Spring Boot/Hibernate, distributed network services, and complex GPS/road-network routing. The project statement describes route distance as an input to the simulation instead of calculating paths on a real map.

4. Functional Requirements

Module	Input	Processing	Output
Rider request processing	Rider ID, name, pickup, drop, distance	Create and start RiderRequestThread; track active demand	Request status and ride record
Driver allocation	Current DriverPool state	Check AVAILABLE drivers and atomically mark selected driver BUSY	Assigned driver or NoDriverAvailableException
Dynamic pricing	Active requests, available drivers, trip distance	Calculate multiplier and fare using bounded ratio	Surge multiplier and final fare
Persistence / audit	Driver and Ride objects	SQLite JDBC persistence and status updates	Stored drivers, ride history, audit query
Reporting	Simulation statistics and persisted rides	Aggregate outcomes and write text report	Console summary and simulation_report.txt
Automated validation	Test scenarios	Execute six assertions across services, concurrency, and DB	PASS/FAIL test summary

The implementation therefore exceeds the minimum requirement of three functional modules: it contains six distinct functional areas. Its I/O is explicit: rider objects enter the simulation, the dispatch layer mutates driver and ride state, and the system produces ride records, console results, and a text audit report.

5. Non-Functional Requirements

Requirement	Implementation evidence	Expected property
Concurrency safety	synchronized DriverPool methods; concurrency test with 10 threads / 3 drivers	No driver is assigned to more than one racing request
Reliability / recovery	finally block releases an assigned driver and decrements active counter	Driver state is restored after normal or interrupted execution

Usability	Structured CLI messages for initialization, requests, dispatch, completion, rejection, audit	Operator can follow a complete simulation run from the terminal
Portability	Java 17 + embedded SQLite; build scripts for Windows and Unix-like systems	No external DB server or network port is required
Maintainability	Separated model, service, DAO, exception, thread, and test packages	Core responsibilities remain modular and isolated
Validation / error handling	InvalidRequestException and NoDriverAvailableException	Invalid input and fleet exhaustion are handled without a process crash
Resource efficiency	Embedded database; short simulated 1.5 second ride duration	Suitable for local coursework demonstrations

6. System Architecture

The project uses a layered, dependency-oriented architecture rather than a framework-heavy stack. Main coordinates the run. Worker threads handle individual rider lifecycles. Services implement the two principal pieces of domain logic: driver allocation and surge pricing. RideDAO isolates database operations. Domain entities and enums represent drivers, riders, rides, and their states. A standalone TestRunner exercises the critical behavior without requiring JUnit.

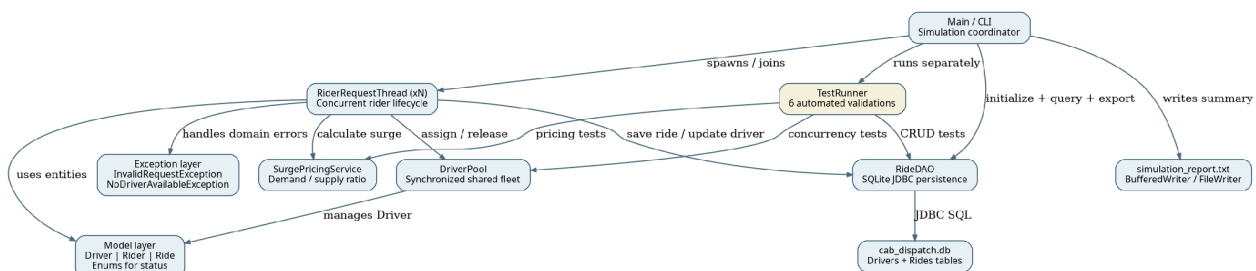


Figure 1. System architecture derived from the repository package structure and component responsibilities.

6.1 Core Components

Component	Responsibility
Main.java	Initializes database/services, seeds three drivers, creates seven rider threads, joins them, aggregates results, queries the audit history, and exports simulation_report.txt.
RiderRequestThread.java	Represents one concurrent request. It validates distance, computes surge and fare, requests a driver, simulates the ride, persists the result, and releases the driver in finally.
DriverPool.java	Stores Driver objects in a HashMap and exposes synchronized operations for adding, assigning, releasing, and counting drivers.

SurgePricingService.java	Calculates a multiplier from activeRequests / availableDrivers and clamps the result to the range 1.0x-3.0x.
RideDAO.java	Initializes the SQLite schema and performs driver/ride persistence using JDBC connections and prepared statements.
TestRunner.java	Runs six assertion-based validations covering pricing, input validation, fleet exhaustion, driver release, concurrency safety, and SQLite persistence.

7. Design Decisions and Rationale

7.1 Synchronized allocation instead of independent synchronization

The critical operation is not merely reading the driver list; it is the compound sequence "find an available driver -> mark it BUSY -> return it." DriverPool therefore synchronizes the entire method. This is important because protecting only the read or only the write would still leave an interleaving window between the two operations.

```
public synchronized Driver assignDriver(String riderName)
    throws NoDriverAvailableException {
    for (Driver driver : drivers.values()) {
        if (driver.getStatus() == DriverStatus.AVAILABLE) {
            driver.setStatus(DriverStatus.BUSY);
            return driver;
        }
    }
    throw new NoDriverAvailableException("No driver available");
}
```

7.2 AtomicInteger for demand telemetry

An AtomicInteger stores the number of active requests because multiple rider threads increment and decrement the value concurrently. The class provides atomic update operations without requiring the counter itself to be protected by the DriverPool monitor.

7.3 Bounded surge pricing

The pricing service uses the ratio activeRequests / availableDrivers. When demand does not exceed supply, the multiplier is 1.0x. When demand exceeds supply, the ratio is used and capped at 3.0x. When supply is zero, the service directly returns the maximum multiplier.

```
Surge = min(max(activeRequests / availableDrivers, 1.0), 3.0)
```

7.4 finally-based resource recovery

The worker thread releases a driver in a finally block. This means the cleanup path is structurally attached to the try/catch region and still executes when a ride finishes normally or when the sleep operation is interrupted. The same block also decrements the active-request counter.

7.5 JDBC with prepared statements

RideDAO isolates database access from the simulation logic. SQL commands are executed using PreparedStatement for mutations, which keeps the parameter binding explicit and avoids constructing the insert statements through string concatenation. SQLite is embedded, so the simulator does not need a separate database server.

8. Implementation Details

8.1 Project Structure

The repository contains 13 Java source files across the main, model, service, dao, exception, thread, and test packages, plus Maven/build configuration and bundled dependency libraries.

Package	Main files	Purpose
com.cabdispatch	Main.java	Application entry point and report generation
model	Driver, Rider, Ride, DriverStatus, RideStatus	Domain state and entities
service	DriverPool, SurgePricingService	Core business logic
dao	RideDAO.java	SQLite/JDBC persistence
exception	2 custom exceptions	Domain-specific failure paths
thread	RiderRequestThread.java	Concurrent request lifecycle
test	TestRunner.java	Standalone automated verification

8.2 Fare Calculation

The current base-fare rule is a fixed flag-drop rate of Rs.50 plus Rs.15 per kilometer. The final fare is the base fare multiplied by the current surge multiplier, rounded to two decimal places. For example, a 3.2 km trip produces a base fare of Rs.98.00; with a 1.33x multiplier the repository sample reports a final fare of Rs.130.34.

8.3 Ride Lifecycle

RideStatus defines REQUESTED, CONFIRMED, COMPLETED, and CANCELLED_NO_DRIVER. The simulation creates completed rides when dispatch succeeds and cancellation records when no driver is available. DriverStatus contains AVAILABLE and BUSY, providing a simple resource state machine.

8.4 Persistence Schema

The SQLite database contains a drivers table with driver_id, name, phone, and status. The rides table stores ride ID, rider details, optional driver ID, locations, distance, base fare, surge multiplier, final fare, status, request time, and completion time. The supplied schema intentionally keeps driver_id nullable so a cancelled request can still be audited without an assigned driver.

9. Screenshots and Results

The repository includes a representative console trace generated by the simulator. Because the application is intentionally CLI-based, the terminal output is the primary user-visible result. The sample run launches seven concurrent rider threads against three drivers, then reports successful and failed requests, surge multipliers, revenue, database audit entries, and file export.

```
CAB DISPATCH & SURGE PRICING SIMULATOR
-----
[INIT] SQLite Database initialized
[SEEDING] 3 drivers registered
LAUNCHING 7 CONCURRENT RIDER THREADS

[DISPATCH SUCCESS] Diya Patel -> D1 | Surge 1.33x | Final Rs.130.34
[DISPATCH SUCCESS] Ananya Verma -> D2 | Surge 1.50x | Final Rs.138.00
[DISPATCH SUCCESS] Kabir Nair -> D3 | Surge 3.00x | Final Rs.375.00
[DISPATCH REJECTED] 4 requests: no driver available

SIMULATION SUMMARY
Total Rider Requests   : 7
Successful Dispatches  : 3
Failed (No Driver)     : 4
Average Surge Applied  : 2.55x
Total Revenue Generated: Rs.643.34

[DATABASE AUDIT] Ride history fetched from SQLite
[FILE I/O] simulation_report.txt generated
```

Figure 7. Representative console result based on the sample output documented in the repository README.

9.1 Representative Metrics

Metric	Documented value	Interpretation
Total rider requests	7	Seven concurrent request threads launched
Available drivers	3	Fleet capacity in the representative scenario
Successful dispatches	3	All three available drivers were assigned

Failed requests	4	Four requests were cancelled due to no available driver
Average surge applied	2.55x	Average across the seven processed requests
Total revenue	Rs.643.34	Sum of final fares for successful rides

The numbers above are taken from the repository-documented sample output. They illustrate the intended behavior of the simulator rather than representing real ride-hailing market data or a benchmark against another dispatch algorithm.

10. Testing Approach

Testing is implemented as a standalone TestRunner so the project does not depend on a unit-test framework. Each test is executed through the same runTest wrapper, which increments counters and reports PASSED or FAILED. The repository documents all six test cases as passed.

#	Test	Scenario / assertion	Repository-documented status
1	Surge Pricing Calculation & Clamping	Checks 1.0x baseline, proportional ratio, 3.0x ceiling, and zero-driver behavior	PASSED
2	Input Validation	Negative ride distance must raise InvalidRequestException	PASSED
3	Fleet Exhaustion	Second assignment from a one-driver busy pool must raise NoDriverAvailableException	PASSED
4	Driver Pool State Recovery	Driver must return from BUSY to AVAILABLE after release	PASSED
5	Concurrency Safety	10 concurrent threads compete for 3 drivers; exactly 3 assignments and 7 rejections; all assigned IDs unique	PASSED
6	SQLite Database Persistence	Create schema, save driver, save ride, retrieve history, verify fare/status	PASSED

10.1 Testing Methodology

1. Run the pricing service against equal, lower, higher, extreme, and zero-driver supply values.
2. Inject invalid rider data to verify checked exception handling.
3. Construct a saturated driver pool to verify graceful fleet exhaustion.
4. Assign and release a driver to verify state recovery.

5. Use `CountDownLatch` so ten threads are released at the same instant, then verify assignment uniqueness.
6. Use a test SQLite database to persist and re-read a driver and a completed ride.

The concurrency test is especially important because correctness is not proven merely by observing one successful simulation run. The dedicated test forces multiple threads to contend at the same time and then checks both capacity and uniqueness invariants.

11. Challenges Addressed

The implementation addresses several practical problems that commonly appear when translating concurrency concepts into working code:

- Race-condition prevention: check-and-assign must be protected as one operation.
- Fleet exhaustion: the simulator needs an explicit exception path instead of crashing when every driver is `BUSY`.
- Resource recovery: drivers must be returned to `AVAILABLE` even when a thread is interrupted.
- Persistent auditability: ride outcomes and driver state changes need to survive beyond in-memory objects.
- Input validation: impossible distances and negative fares are rejected before database mutation.
- Cross-platform execution: the project packages its runtime as a JAR and also provides Windows and Unix-like build scripts.

12. Future Enhancements

The current project is intentionally focused. The following extensions would increase realism or usability without changing its core concurrency design:

- Replace fixed trip distances with real route-distance calculation using a mapping service or graph-based pathfinding.
- Introduce location-aware dispatch so the nearest suitable `AVAILABLE` driver is selected rather than the first available map entry.
- Add a command-line menu or JavaFX/web front end for configuring fleet size, requests, base fare, and simulation duration.
- Persist full driver metadata during ride-history reads rather than reconstructing a generic driver object from only the ID.
- Add stronger relational constraints, including an explicit foreign key for `rides.driver_id` where supported by the chosen schema policy.
- Add structured logging, timestamps with richer metadata, and export formats such as CSV/JSON.
- Expand the test suite with repeated stress runs and configurable thread/fleet sizes.
- Experiment with queueing and request-retry policies so a temporarily unavailable request can wait rather than being immediately cancelled.

13. Conclusion

The Cab Dispatch & Surge Pricing Simulator is a compact end-to-end Java project that combines object-oriented modeling, multithreading, synchronization, custom exceptions, JDBC persistence, file I/O, and automated testing. Its most important design property is the synchronized DriverPool: the system treats the availability check and status change as one critical section, which is the key mechanism used to prevent double-booking.

The simulator also shows how application logic, persistence, reporting, and validation can be integrated without introducing a heavyweight framework. The documented sample run and six-test harness provide concrete evidence of the intended behaviors. At the same time, the project remains a simulation and should not be interpreted as a production ride-hailing system; geographical dispatch, network services, authentication, and real operational demand data are outside its scope.

14. References

1. VITyarthi - Build Your Own Project: General Project Instructions & Submission Guidelines.
2. ShubratJain-VITBhopal, [cab-dispatch-simulator](#) (repository reviewed 18 September 2026).
3. Project source files and documentation: README.md, statement.md, Java source packages, pom.xml, and included build/test configuration in the repository.

Source Review Note

This report was prepared by reviewing the repository documentation and source files available through the public GitHub repository on 18 September 2026, together with the supplied VITyarthi project-report requirements and the supplied reference cover page. The numerical simulation output and test results are reported as documented in the repository; they were not independently executed in a local clone during report preparation.